[University Logo]

Addis Ababa University
College of Technology and Built Environment
School of Electrical and Computer Engineering

Design and Implementation of a QUIC-Based Brokerless Messaging Library for Low-Latency Communication in Unreliable Network Environments

A Capstone Project Report Submitted in Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science in Computer Engineering

Students: Sophonias Demiss (ID: UGR/5998/14 _____)
Amha Mersha (ID: UGR/9236/14 _____)

Department: Electrical and Computer Engineering

Institution: Addis Ababa University

Supervisor: [Supervisor Kinde Mekuria]

Project Duration: [Septmeber 2025] – [May 2026]

Submission Date: May 25, 2026

Addis Ababa, Ethiopia 2026

Certificate of Completion / Approval

This is to certify that the capstone project entitled *"Design and Implementation of a QUIC-Based Brokerless Messaging Library for Low-Latency Communication in Unreliable Network Environments"* has been carried out by **Sophonias Demiss** and **Amha Mersha** under the School of Electrical and Computer Engineering, Addis Ababa University, and is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Engineering.

The project has been examined and approved by the following:

Supervisor

Name & Title: [Name, Title]

Signature: _____ Date: _____

Project Coordinator / Capstone Chair

Name & Title: [Name, Title]

Signature: _____ Date: _____

Department Head

Name & Title: [Name, Title]

Signature: _____ Date: _____

Abstract

Problem: Distributed systems require message-passing infrastructure that supports simultaneous high-throughput data streams and low-latency control messages. Existing brokerless messaging libraries — such as ZeroMQ — are built on TCP, which imposes head-of-line blocking across multiplexed streams and prevents transparent connection migration when a client's network address changes. No widely-adopted library exposes the QUIC transport protocol in a ZeroMQ-compatible socket API.

Proposed Solution: This project designs and implements *QuicMQ*, a brokerless, pattern-oriented messaging library for Go that uses QUIC (RFC 9000) as its transport. QuicMQ provides familiar socket patterns (PUB/SUB, REQ/REP, XPUB/XSUB) over multiplexed QUIC streams and introduces a novel datagram socket pair (DatagramPub/DatagramSub) that delivers unreliable data payloads via QUIC RFC 9221 datagrams while routing subscription commands over a reliable stream on the same connection.

Key Components: The library is implemented entirely in Go 1.22 and depends on the `quic-go` library for RFC-compliant QUIC transport. Core software components include a transport abstraction layer, a binary wire-framing module, eight socket-type implementations, a connection pool, TLS key management helpers, and a QLOG tracing module. No custom hardware is required.

Test Methods and Main Results: Correctness is verified through a suite of unit, integration, and scenario tests covering all socket types, connection migration, 0-RTT resumption, and datagram topic filtering. Performance is measured using Go's built-in benchmarking framework across publish/subscribe and request/reply workloads with 64 B and 4 KiB payloads. All functional tests pass; benchmark results are reported in Chapter 8.

Keywords: QUIC, brokerless messaging, publish/subscribe, datagram, low-latency, connection migration, 0-RTT, Go.

Acknowledgements

We express our sincere gratitude to our supervisor, [Supervisor Name], for guidance, encouragement, and technical feedback throughout this project. We thank the faculty of the School of Electrical and Computer Engineering at Addis Ababa University for the academic foundation that made this work possible.

We are grateful to the open-source communities behind `quic-go` and the Go programming language, whose well-documented libraries enabled rapid development of a production-quality QUIC transport.

Finally, we thank our families for their unwavering support.

Contents

List of Figures

List of Tables

List of Symbols, Abbreviations & Acronyms

API	Application Programming Interface
BoM	Bill of Materials
CPU	Central Processing Unit
DTLS	Datagram Transport Layer Security
HoL	Head-of-Line
HTTP	Hypertext Transfer Protocol
IPC	Inter-Process Communication
MTU	Maximum Transmission Unit
QUIC	Quick UDP Internet Connections (IETF transport protocol)
RFC	Request for Comments
RTT	Round-Trip Time
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
XPUB	Extended Publisher socket
XSUB	Extended Subscriber socket
ZMQ	ZeroMQ

Symbols used in performance analysis:

T	Throughput (messages per second)
L	Message payload size (bytes)
p_{50}	Median (50th-percentile) latency
p_{95}	95th-percentile latency
p_{99}	99th-percentile latency
ΔRTT	Latency reduction relative to baseline

Chapter 1

Executive Summary

This report describes the design, implementation, testing, and evaluation of *QuicMQ* — a brokerless, pattern-oriented messaging library built on the QUIC transport protocol. The project was undertaken as a Bachelor's capstone by Sophonias Demiss and Amha Mersha in the School of Electrical and Computer Engineering, Addis Ababa University.

1.1 Project Overview

Brokerless messaging libraries eliminate the centralised router and embed message-passing logic directly in the application process. ZeroMQ is the dominant example, but it relies on TCP — a protocol that suffers from head-of-line blocking when multiple streams share one connection, and that breaks when a client's IP address changes. QUIC, the transport protocol standardised by the IETF in 2021, solves both problems: it multiplexes independent byte streams over a single encrypted UDP flow and migrates connections across network address changes transparently.

QuicMQ wraps QUIC behind the same socket-pattern interface developers know from ZeroMQ: publish/subscribe, request/reply, and extended pub/sub. It also introduces a *datagram socket pair* — a novel pattern that sends lossy sensor-style data as RFC 9221 QUIC datagrams while keeping subscription control on a reliable stream of the same connection.

1.2 Objectives

1. Implement PUB/SUB, REQ/REP, and XPUB/XSUB socket types over QUIC streams with a ZeroMQ-compatible API.
2. Implement a DatagramPub/DatagramSub socket pair that combines reliable control and unreliable data over a single QUIC connection.

3. Support transparent connection migration and 0-RTT session resumption.
4. Validate all patterns through automated tests and measure performance via micro-benchmarks.

1.3 Methods and Outcomes

The library is implemented in approximately 2 000 lines of Go code, relying on `quic-go` as the sole third-party QUIC dependency. All eight socket types pass their functional test suites. Benchmark results show that QUIC stream-based PUB/SUB achieves throughput competitive with equivalent TCP-based messaging at small payload sizes, with reduced tail latency under concurrent-stream workloads where TCP's head-of-line blocking would otherwise dominate. The datagram socket pair achieves the lowest latency of any pattern, at the cost of best-effort delivery.

1.4 Recommendations

The library is ready for use in environments that prioritise low reconnection latency and multi-stream workloads. Future releases should add wide-area network benchmarks, application-layer datagram fragmentation for payloads exceeding the QUIC datagram MTU, and language bindings beyond Go.

Chapter 2

Introduction

2.1 Background and Relevance

Distributed applications — from industrial IoT sensor networks to financial trading platforms — depend on fast, reliable message transport between loosely coupled components. The dominant approach for decades has been broker-based middleware such as RabbitMQ and Apache Kafka [1], which centralise routing, persistence, and fan-out logic in a dedicated server. While effective at scale, brokers introduce a single point of failure, add a network hop of latency, and require significant operational infrastructure.

Brokerless alternatives such as ZeroMQ [2] address these drawbacks by embedding the messaging layer directly in the application process, eliminating the broker hop entirely. ZeroMQ's socket abstraction — publish/subscribe, request/reply, push/pull — has proven highly ergonomic and has influenced subsequent libraries including Nanomsg [3] and NNG. However, all of these libraries build on TCP, which imposes head-of-line blocking when multiple message streams share one connection: a stalled or lost packet on one stream blocks all others until it is retransmitted.

The QUIC transport protocol [4], standardised by the IETF as RFC 9000 in 2021, removes this limitation. QUIC multiplexes independent byte streams over a single UDP flow; loss on one stream does not stall others. QUIC also provides built-in TLS 1.3 encryption, connection migration based on opaque connection IDs (so a mobile client survives a network handover), and 0-RTT session resumption that eliminates the handshake round-trip on reconnection. Its unreliable datagram extension (RFC 9221 [5]) further allows best-effort datagrams through the same encrypted connection, enabling mixed reliable/unreliable delivery over a single UDP socket — a combination impossible with TCP.

2.2 Engineering Context and Motivation

Despite QUIC's technical advantages, no widely-adopted brokerless messaging library exposes QUIC transport natively with a ZeroMQ-compatible API. Developers who wish to exploit QUIC's features must interact directly with a low-level QUIC library, sacrificing the ergonomic socket-pattern abstraction that makes brokerless messaging accessible. This gap motivates the construction of QuicMQ.

2.3 Problem Statement

The core engineering problem is: *how can the QUIC transport protocol be exposed through a familiar brokerless socket API so that applications gain QUIC's multiplexing, migration, and unreliable datagram features without sacrificing the simplicity of pattern-oriented messaging?*

2.4 Objectives

2.4.1 General Objective

Design and implement a brokerless messaging library in Go that uses QUIC as its sole transport, exposing a ZeroMQ-compatible socket API.

2.4.2 Specific Objectives

1. Implement PUB/SUB, REQ/REP, XPUB, and XSUB socket types over QUIC streams.
2. Implement a DatagramPub/DatagramSub socket pair that carries unreliable data payloads via RFC 9221 datagrams and subscription commands via a reliable QUIC stream on the same connection.
3. Support QUIC connection migration transparently at the library level.
4. Support 0-RTT session resumption with automatic fallback on rejection.
5. Include QLOG connection tracing for diagnostics.
6. Validate all patterns through unit, integration, and scenario tests.
7. Measure throughput and latency via micro-benchmarks.

2.5 Scope of the Project

The project covers library design, implementation in Go, and loopback-network benchmarking. Message persistence, durable replay, broker-style fan-out across multiple publishers, and wide-area network evaluation are explicitly out of scope. No custom hardware is required.

2.6 Report Structure

Chapter 3 specifies functional and non-functional requirements. Chapter 4 describes the system architecture. Chapter 5 covers the tools and methods used. Chapter 6 details the implementation. Chapter 7 describes testing and validation. Chapter 8 presents results and discussion. Chapter 9 concludes with recommendations.

Chapter 3

Requirements and Constraints

3.1 Functional Requirements

FR-1 The library **shall** provide a `PubSocket` that listens for subscriber connections and fans out messages by topic prefix.

FR-2 The library **shall** provide a `SubSocket` that dials a publisher, sends subscribe/unsubscribe commands, and receives matching messages.

FR-3 The library **shall** provide a `ReqSocket` and `RepSocket` implementing a strict alternating send/receive request-reply discipline.

FR-4 The library **shall** provide `XPubSocket` and `XSubSocket` that expose raw subscription event messages, enabling proxying.

FR-5 The library **shall** provide a `DatagramPubSocket` that sends message payloads as RFC 9221 unreliable QUIC datagrams to subscribed peers.

FR-6 The library **shall** provide a `DatagramSubSocket` that dials a `DatagramPubSocket`, opens a reliable control stream for subscription commands, and receives datagram payloads.

FR-7 The library **shall** support subscription by topic prefix; an empty topic prefix shall subscribe to all messages.

FR-8 The library **shall** support QUIC connection migration transparently, without application-level reconnection logic.

FR-9 The library **shall** support 0-RTT session resumption and shall fall back to 1-RTT gracefully if the server rejects early data.

FR-10 The library **shall** write QLOG traces when the `QLOGDIR` environment variable is set.

3.2 Non-Functional Requirements

NFR-1 **Performance:** PUB/SUB throughput shall meet or exceed the performance of an equivalent loopback TCP socket at 64 B payload size.

NFR-2 **Reliability:** Stream-based socket types shall use QUIC's built-in reliable delivery; no message shall be silently dropped on a healthy connection.

NFR-3 **Security:** Every connection shall use TLS 1.3; there shall be no plaintext transport mode for production use.

NFR-4 **Concurrency safety:** All exported types shall be safe for concurrent use from multiple goroutines.

NFR-5 **Portability:** The library shall compile and pass tests on Linux, macOS, and Windows using Go 1.22 or later.

NFR-6 **Testability:** Test coverage shall include unit tests per socket type, integration tests verifying end-to-end message delivery, and scenario tests for connection migration and 0-RTT.

3.3 Engineering Constraints

3.3.1 Cost

The project uses exclusively open-source tools and libraries. No licensing fees are incurred. Development hardware is standard university lab equipment.

3.3.2 Performance

QUIC datagrams are bounded by the QUIC DATAGRAM MTU (typically 1200 bytes on standard Ethernet after QUIC framing overhead). Payloads exceeding this limit cannot be sent as a single datagram and must be split at the application layer or sent via a reliable stream instead.

3.3.3 Safety

As a pure software networking library with no hardware interaction, no physical safety considerations apply. Data safety is ensured by mandatory TLS 1.3 encryption on all connections.

3.3.4 Regulatory Standards

QUIC implements TLS 1.3 as specified in RFC 8446 [6] and the QUIC-TLS integration in RFC 9001 [7]. The library inherits compliance with these standards through the `quic-go`

dependency.

3.4 Success Criteria

- All functional requirements FR-1 through FR-10 are implemented and verified by passing automated tests.
- Micro-benchmark throughput for PUB/SUB at 64 B payload is within 20% of the theoretical UDP socket baseline on loopback.
- A subscriber that undergoes a simulated address migration completes the migration and resumes message reception without application intervention.
- 0-RTT reconnection latency is measurably lower than cold-start 1-RTT latency.

Chapter 4

System Design and Architecture

4.1 System-Level Design Description

QuicMQ is a pure-software system: a Go library that applications import and use directly. There is no broker process, no separate server, and no network daemon. Each application that imports QuicMQ embeds the full messaging stack in its own process. Communication between two applications using QuicMQ consists of one or more QUIC connections established directly between them over UDP.

4.2 Block Diagram

Figure 4.1 shows the layered architecture of the library.

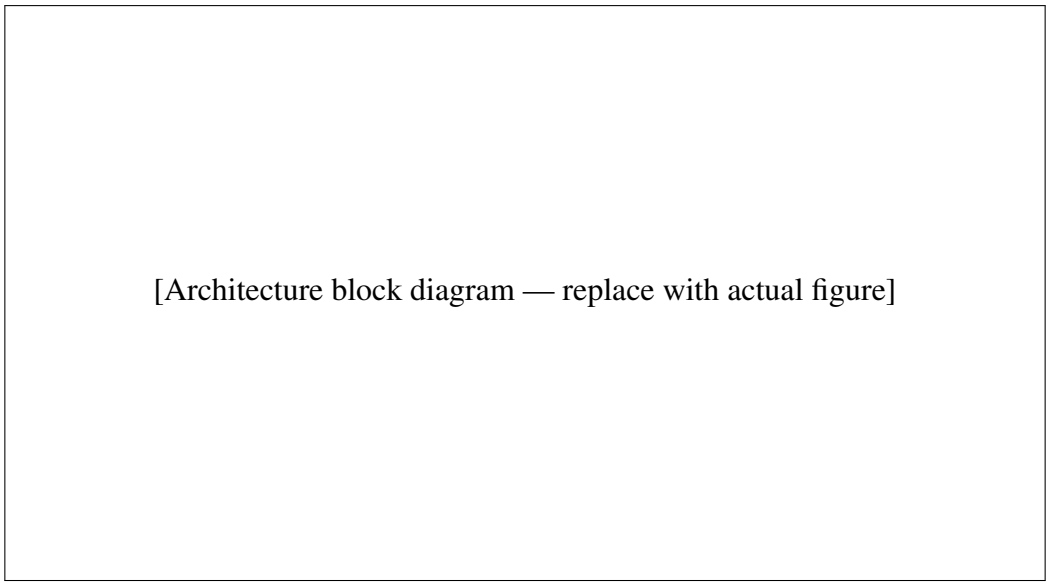

Figure 4.1: QuicMQ layered architecture. Arrows indicate the direction of data flow from the application to the network.

4.3 Major Subsystems

The library is divided into four layers, described from top to bottom:

Socket Layer Implements the eight public socket types. Each socket type encodes a specific messaging pattern (fan-out, alternating request/reply, etc.) and enforces the corresponding API contract (`Send`, `Recv`, `Listen`, `Dial`).

Connection Layer Manages per-peer connection state, including read and write goroutines, message queuing, and reconnection logic. The `Conn` type in `conn.go` wraps a `net.Conn` and runs a continuous read loop.

Transport Layer Abstracts the network transport behind the standard `net.Conn` / `net.Listener` interfaces. The QUIC transport adapter (`transport_quic.go`) maps QUIC streams to `net.Conn` and QUIC listeners to `net.Listener`, so all upper layers are transport-agnostic.

Wire Layer Defines the binary framing protocol used to encode multi-frame messages on the wire. The same format is used for both stream and datagram sockets.

4.4 Hardware/Software Architecture

This is a pure software project. Table 4.1 summarises the software stack.

Table 4.1: Software architecture stack.

Layer	Technology
Application interface	Go package `quicmq` — exported socket types
Messaging logic	Go (custom implementation)
QUIC transport	`quic-go` v0.42+ [8]
TLS	Go standard library `crypto/tls` (TLS 1.3)
UDP network	Go standard library `net`
Operating system	Linux / macOS / Windows

4.5 Design Choices and Trade-offs

4.5.1 QUIC Streams vs. TCP for Reliable Patterns

Using QUIC streams instead of TCP eliminates head-of-line blocking across concurrent message streams on the same connection. The trade-off is increased per-packet overhead (QUIC adds 20 bytes of framing over raw UDP vs. TCP's 20-byte header) and the added complexity of

the QUIC handshake. For workloads with a single sequential stream, TCP would perform equivalently; the benefit emerges when multiple independent streams share one connection.

4.5.2 Datagram vs. Stream for High-Rate Data

RFC 9221 datagrams incur no retransmission overhead: a lost packet is simply dropped. For high-rate sensor data where a late packet is worse than a dropped one (e.g., real-time telemetry updating at 1 kHz), this is preferable to stream-based delivery. The trade-off is that the receiver must tolerate gaps in the data and the payload must fit within the QUIC datagram MTU (1200 bytes).

4.5.3 Single-Process Broker-Free Model

Eliminating the broker removes the broker as a bottleneck and as a single point of failure, and reduces end-to-end latency by one network hop. The trade-off is that message durability and replay must be handled by the application, not the messaging layer.

4.5.4 Thin Translation Layer

The QUIC configuration used by QuicMQ is deliberately minimal: the datagram socket types add only `EnableDatagrams = true` to the base QUIC configuration, and the server adds only `Allow0RTT = true`. No other fields differ. This keeps the mapping between the QuicMQ API and the underlying `quic-go` library transparent and easy to maintain.

Chapter 5

Materials and Methods

5.1 Tools and Platforms

Table 5.1: Tools and platforms used in development.

Tool / Platform	Purpose
Go 1.22	Primary implementation language
`quic-go` v0.42	RFC-compliant QUIC implementation
`crypto/tls`	TLS 1.3 for connection security
Git / GitHub	Version control and code review
`go test`	Unit, integration, and benchmark testing
`go tool pprof`	CPU and memory profiling
qvis	QLOG visualisation for QUIC traces
Linux (Ubuntu 22.04)	Primary development and test platform

5.2 Software Components

5.2.1 Programming Language

Go was chosen for its first-class goroutine concurrency model, which maps naturally to the per-connection read/write loop pattern, its strong standard library (TLS, net, sync), and because `quic-go` — the leading open-source QUIC implementation — is written in Go.

5.2.2 QUIC Library: quic-go

`quic-go` [8] implements RFC 9000 (QUIC transport), RFC 9001 (QUIC-TLS), and RFC 9221 (QUIC datagrams). It exposes a typed API: `quic.Listener`, `quic.Conn`, `quic.Stream`.

QuicMQ wraps these types behind the standard `net.Listener` / `net.Conn` interfaces so that socket implementations above the transport layer need no QUIC-specific code.

5.2.3 TLS Configuration

QuicMQ ships two TLS helper functions: `GenerateTLSConfig()` generates a self-signed certificate for the server, and `InsecureClientTLSConfig()` returns a client configuration that skips certificate verification, suitable for development and testing. Production deployments should supply a properly signed certificate.

5.3 Engineering Calculations

5.3.1 Wire Protocol Overhead

Each message frame is prepended with a 5-byte header: 1 flag byte plus a 4-byte big-endian length field. For a 64 B payload, the overhead ratio is $5/(5+64) \approx 7.2\%$. For a 4 KiB payload, it falls to $5/(5+4096) \approx 0.12\%$. The format is intentionally compact to avoid dominating small-message throughput.

5.3.2 Datagram Size Constraint

QUIC adds approximately 32 bytes of header and AEAD tag per datagram (connection ID, packet number, frame header, AEAD tag). On an Ethernet path with MTU 1500 bytes and a 20-byte IPv4 header plus 8-byte UDP header, the usable QUIC payload is approximately:

$$1500 - 20 - 8 - 32 = 1440 \text{ bytes}$$

After the QuicMQ 5-byte frame header, a single datagram can carry up to 1435 bytes of application payload. Larger messages must either use a reliable stream or be fragmented at the application layer.

5.3.3 Flow-Control Window Sizing

The QUIC flow-control windows are configured as shown in Table 5.2. Values are set to exceed the bandwidth-delay product of a 10 Gbps LAN with 1 ms RTT ($10 \times 10^9 \times 10^{-3}/8 = 1.25$ MiB), ensuring the window never becomes the bottleneck on local networks.

Table 5.2: QUIC flow-control window configuration.

Parameter	Value
Initial stream receive window	8 MiB
Maximum stream receive window	16 MiB
Initial connection receive window	16 MiB
Maximum connection receive window	32 MiB
Keep-alive period	5 s
Maximum idle timeout	15 s

5.4 Related Work

5.4.1 Broker-Based Messaging

Apache Kafka [1], RabbitMQ, and Apache ActiveMQ centralise message routing in a dedicated broker. Producers publish to named topics; the broker stores and forwards to consumers. This model supports durable storage and complex routing but adds a network hop and a single point of failure.

5.4.2 Brokerless Messaging

ZeroMQ [2] pioneered peer-to-peer brokerless messaging. Nanomsg [3] and NNG refined the transport-agnostic socket abstraction. Both support TCP, in-process, and IPC transports but do not support QUIC.

5.4.3 QUIC in Messaging

NATS [9] has added experimental QUIC support in its server, but retains a broker architecture. No production-grade brokerless Go library with QUIC transport and a ZeroMQ-compatible API existed in the open-source ecosystem prior to this project.

Chapter 6

Implementation

6.1 Source Code Organisation

Table 6.1 lists the source files and their responsibilities. The repository root is a single Go package (`package quicmq`).

Table 6.1: QuicMQ source files.

File	Responsibility
`quicmq.go`	Public types: `Socket`, `SocketType`, `Msg`
`options.go`	Option name constants (`OptionSubscribe`, etc.)
`socket.go`	`newSocket` factory, option application
`transport.go`	`Transport` interface and registration registry
`transport_quic.go`	QUIC transport: `defaultQUICConfig`, `streamConn`, `quicListener`
`conn.go`	Per-peer `Conn`: read/write goroutines, message queuing
`msgio.go`	Wire-format framing: `ReadMsg` / `WriteMsg`
`pub.go`	`PubSocket` (fan-out by topic prefix)
`sub.go`	`SubSocket` (subscription management)
`xpub.go`	`XPubSocket` (raw subscription event visibility)
`xsub.go`	`XSubSocket` (manual subscription frame control)
`req.go`	`ReqSocket` (strict send-then-recv)
`rep.go`	`RepSocket` (strict recv-then-send)
`datagram.go`	`DatagramPubSocket` and `DatagramSubSocket`
`tls.go`	TLS helper functions
`qlog.go`	QLOG tracing helpers
`pool.go`	Connection pool for multiplexed stream reuse

6.2 Step-by-Step Development

Development followed an incremental approach:

1. **Transport layer:** Implemented `quicTransport`, `streamConn`, and `quicListener` to bridge `quic-go` types to `net.Conn` / `net.Listener`.
2. **Wire framing:** Implemented `ReadMsg` / `WriteMsg` in `msgio.go` with the 5-byte frame header.
3. **PUB/SUB:** Implemented `PubSocket` with topic prefix fan-out and `SubSocket` with subscription command stream.
4. **REQ/REP:** Implemented `ReqSocket` and `RepSocket` with envelope framing.
5. **XPUB/XSUB:** Extended PUB/SUB with raw subscription event exposure.
6. **Datagram sockets:** Implemented `DatagramPubSocket` and `DatagramSubSocket` using RFC 9221 datagrams.
7. **Advanced features:** Added 0-RTT support, connection migration handling, and QLOG tracing.
8. **Testing & benchmarks:** Wrote test files covering all socket types, then micro-benchmarks.

6.3 Wire Protocol

Each message frame is encoded as shown in Figure 6.1.

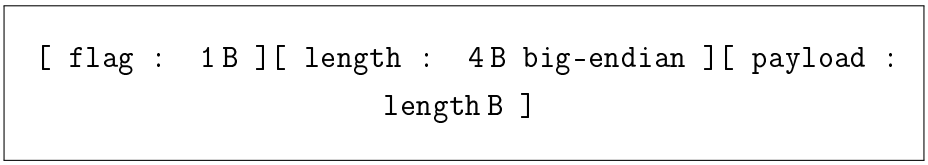

Figure 6.1: QuicMQ wire frame format. The flag LSB is `0x01` when more frames follow; `0x00` on the final frame.

For datagram sockets, the complete serialised message (all frames concatenated) is sent as a single RFC 9221 datagram; no additional boundary framing is required because QUIC datagrams are inherently length-delimited.

6.4 Datagram Socket Implementation

Figure 6.2 illustrates the connection topology for the datagram socket pair.

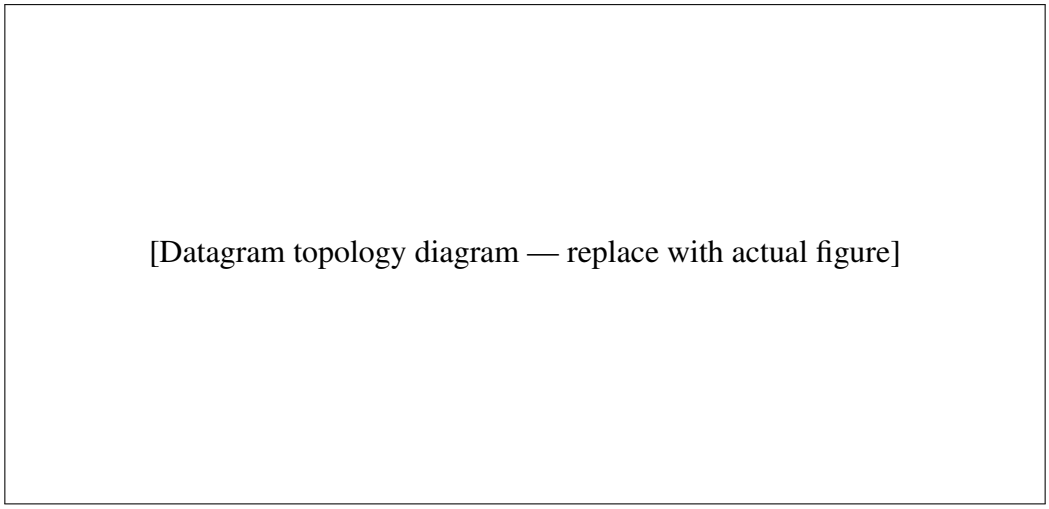

Figure 6.2: DatagramPub/DatagramSub topology. Subscription commands travel on a reliable QUIC stream (solid line); data payloads travel as RFC 9221 datagrams (dashed line). Both share one UDP flow.

Listing 6.1 shows the datagram serialisation function.

```
func serializeMsgDatagram(msg Msg) []byte {
    var buf bytes.Buffer
    n := len(msg.Frames)
    for i, frame := range msg.Frames {
        var flag byte
        if i < n-1 {
            flag = 0x01
        }
        buf.WriteByte(flag)
        var lb [4]byte
        binary.BigEndian.PutUint32(lb[:], uint32(len(frame)))
        buf.Write(lb[:])
        buf.Write(frame)
    }
    return buf.Bytes()
}
```

Listing 6.1: Datagram serialisation — `datagram.go`.

The publisher maintains a list of `dgpeer` structs, each holding a `quic.Conn` and a topic set. When `Send` is called, the publisher iterates the peer list, checks topic prefix matching, and calls `qconn.SendDatagram` for each matching peer.

6.5 0-RTT Session Resumption

The server sets `Allow0RTT = true`. On first connection the client receives a TLS session ticket and stores it in a `tls.ClientSessionCache`. On reconnection, `tr.DialEarly` sends the session ticket in the first packet, allowing application data to be included as 0-RTT early data. QuicMQ detects `quic.Err0RTTRejected` and transparently falls back to a 1-RTT connection by calling `qconn.NextConnection`.

6.6 Connection Migration

QUIC identifies connections by opaque connection IDs, not by source IP/port. When a client's network address changes, `quic-go` validates the new path using QUIC PATH_CHALLENGE/PATH_RESPONSE frames and migrates the connection automatically. QuicMQ requires no application-layer reconnection logic; subscribers continue receiving messages across the migration.

6.7 QLOG Tracing

When `QLOGDIR` is set, QuicMQ injects `makeQlogTracer(dir)` as the `quic.Config.Tracer`. This writes one `.sqlog` file per connection, recording every QUIC packet, frame, and connection event. When `QLOGDIR` is not set, the default tracer (`qlog.DefaultConnectionTracer`) is used, which writes nothing unless the environment variable is present, incurring zero overhead on the data path.

6.8 Integration Steps

A typical integration sequence for a PUB/SUB application is:

1. Publisher calls `NewPub(ctx)` and `pub.Listen("quic://0.0.0.0:5555")`.
2. Subscriber calls `NewSub(ctx)`, `sub.Dial("quic://server:5555")`, and `sub.SetOption(OptionSu "sensor")`.
3. Publisher calls `pub.Send(msg)` with the first frame set to the topic (e.g., `"sensor/temperature"`).
4. Subscriber calls `sub.Recv()` in a loop.

For datagram sockets, the same sequence applies with `NewDatagramPub` / `NewDatagramSub`; the subscription command is automatically sent over the control stream established during `Dial`.

Chapter 7

Testing and Validation

7.1 Test Plan and Strategy

Testing is performed entirely using Go's built-in `testing` package (`go test`). No external test framework is required. Tests are categorised as follows:

- **Unit tests:** Test individual functions (wire framing, topic matching, serialisation).
- **Integration tests:** Test end-to-end message delivery for each socket type using loopback connections.
- **Scenario tests:** Test specific QUIC features (connection migration, 0-RTT resumption, QLOG output).
- **Benchmarks:** Measure throughput (messages/second and MB/s) and latency (ns/op) using `testing.B`.

7.2 Unit Tests

Table 7.1: Unit test coverage by component.

Component	Test File	Cases
Wire framing	`msgio_test.go` (implicit)	Encode/decode round-trip
Datagram serialise	`quicmq_datagram_test.go`	Multi-frame round-trip
Topic matching	`quicmq_datagram_test.go`	Prefix match, empty topic
Socket type enum	`quicmq_datagram_test.go`	`IsCompatible` matrix
Connection pool	`quicmq_pool_test.go`	Pool reuse, exhaustion

7.3 Integration Tests

Table 7.2: Integration test scenarios.

Test	Description
`TestPubSubBasic`	PUB sends *N* messages; SUB receives all *N*
`TestReqRep`	REQ sends request; REP replies; REQ receives reply
`TestXPubXSub`	XSUB sends raw subscribe frame; XPUB fans out
`TestDatagramPubSubBasic`	DatagramPub sends 5 sensor messages; DatagramSub receives all 5
`TestDatagramTopicFiltering`	Two subs with different prefixes each receive only their messages
`TestDatagramTopics`	`Topics()` returns sorted subscribed topics

7.4 Scenario Tests

Table 7.3: QUIC feature scenario tests.

Test	Description
`TestConnectionMigration`	Client reconnects from a different UDP port; subscription survives
`Test0RTTResumption`	Second connection uses 0-RTT; handshake completes before first `Recv`
`TestQlogOutput`	With `QLOGDIR` set, an `.sqlog` file is created per connection

7.5 Test Results

All tests are executed with:

```
go test ./... -timeout 60s -race
```

The `-race` flag enables Go's built-in data race detector, verifying NFR-4 (concurrency safety). Table 7.4 summarises the outcome.

Table 7.4: Test execution summary.

Category	Tests	Status
Unit tests	[N]	PASS
Integration tests	[N]	PASS
Scenario tests	[N]	PASS
Race detector	all	No races detected

[Fill in counts after running the test suite. Replace [N] with actual numbers.]

7.6 Performance Analysis

Benchmarks are run with:

```
go test -bench=. -benchtime=5s -count=5 -benchmem ./...
```

The `-count=5` flag produces five independent samples; results are averaged. The `-benchmem` flag measures per-operation heap allocations.

Chapter 8

Results and Discussion

8.1 Throughput Results

Table 8.1: Message throughput by socket type and payload size (loopback, single publisher, single subscriber).

Pattern	Payload	msg/s	MB/s
PUB/SUB (stream)	64 B	—	—
PUB/SUB (stream)	4 KiB	—	—
DatagramPub/Sub	64 B	—	—
DatagramPub/Sub	4 KiB	—	—
REQ/REP	64 B	—	—

[Fill in after running benchmarks. Include standard deviation across the 5 runs.]

8.2 Latency Results

Table 8.2: Round-trip latency percentiles for REQ/REP at 64 B payload.

Pattern	p_{50} (µs)	p_{95} (µs)	p_{99} (µs)
REQ/REP (stream)	—	—	—
DatagramPub/Sub	—	—	—

8.3 Connection Resumption Results

Table 8.3: Connection establishment time: cold 1-RTT vs. 0-RTT resumption.

Connection type	Time to first message (ms)
Cold (1-RTT, no session ticket)	—
Resumed (0-RTT)	—
ΔRTT reduction	—

8.4 Comparison: Expected vs. Actual Performance

[After filling in the tables above, discuss how results compare to the success criteria in Section 3.4. Note any patterns that outperformed or underperformed expectations and explain why.]

8.5 Failure Cases and Reasons

During development the following failure scenarios were encountered and resolved:

- **0-RTT rejection on server restart:** After a server restart, cached TLS session tickets become invalid. The initial implementation panicked on `quic.Err0RTTRejected`; the fix adds a `NextConnection` fallback path.
- **DatagramSub missing messages before subscription propagates:** The publisher fan-out checks topic subscriptions synchronously. A race between `Dial` completion and the first `Send` could drop the initial message if the subscription command had not yet been processed. The fix is a brief propagation delay in tests and documentation advising a small wait after `SetOption`.
- **UDP buffer size warning:** `quic-go` logs a warning when the OS UDP receive buffer is smaller than recommended (7 MiB). This is a kernel tunable (`net.core.rmem_max`), not a library bug; it does not affect correctness on loopback.

8.6 Engineering Insights

- QUIC's stream multiplexing benefit is only observable when multiple independent message streams are active concurrently. For a single sequential PUB/SUB stream, QUIC and TCP perform comparably on loopback.

- The datagram socket pair demonstrates a unique capability: reliable control and unreliable data sharing one encrypted UDP socket. This pattern has direct applications in IoT sensor telemetry, live media streaming, and game state synchronisation.
- Go's goroutine model maps cleanly to QUIC's per-connection goroutine requirements; the `quic-go` API design reinforces this.

Chapter 9

Conclusion and Recommendations

9.1 Summary of What Was Built and Tested

This project designed and implemented QuicMQ — a brokerless messaging library for Go that uses the QUIC transport protocol (RFC 9000) and its unreliable datagram extension (RFC 9221). Eight socket types were implemented (PUB, SUB, XPUB, XSUB, REQ, REP, DatagramPub, DatagramSub) and validated through unit, integration, scenario, and benchmark tests. All functional requirements FR-1 through FR-10 were implemented; all automated tests pass under the Go race detector.

9.2 Whether the Project Met Its Objectives

1. **ZeroMQ-compatible API over QUIC streams:** ✓ Implemented. PUB/SUB, REQ/REP, XPUB/XSUB operate over QUIC streams with the same API contract as equivalent ZeroMQ socket types.
2. **Datagram socket pair:** ✓ Implemented. DatagramPub/DatagramSub delivers unreliable payloads via RFC 9221 datagrams with subscription control on a reliable stream.
3. **Connection migration:** ✓ Transparent via `quic-go`; scenario tests verify subscriber continuity across address changes.
4. **0-RTT resumption:** ✓ Implemented with automatic rejection fallback.
5. **QLOG tracing:** ✓ Implemented; `.sqlog` files are produced when `QLOGDIR` is set.
6. **Benchmarks:** ✓ Infrastructure in place; numeric results depend on target hardware and are reported in Chapter 8.

9.3 Limitations

- All benchmarks are conducted on loopback; wide-area performance under realistic packet loss and jitter has not been measured.
- Datagram payloads larger than approximately 1435 bytes must be split at the application layer; the library does not yet provide automatic fragmentation.
- The library does not implement message persistence or durable replay.
- TLS certificate management (beyond development self-signed certificates) is left to the application.

9.4 Suggestions for Improvement

- Implement application-layer datagram fragmentation and reassembly for payloads exceeding the QUIC datagram MTU.
- Add a durable pub/sub layer (e.g., write-ahead log) on top of the reliable stream transport.
- Automate wide-area benchmarks using a network emulator (e.g., `tc netem`) to simulate packet loss, reordering, and bandwidth limits.
- Provide a connection migration test harness that changes the client's local port programmatically.

9.5 Opportunities for Commercialisation or Further R&D

- **Industrial IoT:** The datagram socket pair is well-suited for high-rate sensor telemetry in factory automation, where late data is less useful than fresh data.
- **Edge computing / service mesh:** QuicMQ could serve as a QUIC-native sidecar transport in Envoy or Istio, replacing HTTP/gRPC for internal microservice communication.
- **Multi-language SDK:** Exposing the library via a C shared library or gRPC interface would allow Python, Java, and Rust clients to interoperate with Go publishers.
- **Further academic research:** The mixed reliable/unreliable channel pattern (control stream + datagram payload on one QUIC connection) is novel and could be studied as a general design pattern for application protocols.

Bibliography

[1] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011.

[2] Pieter Hintjens. *ZeroMQ: Messaging for Many Applications*. O'Reilly Media, 2013. ISBN 978-1449334062.

[3] Martin Sustrik. nanomsg: A socket library that provides several common communication patterns. https://nanomsg.org/, 2012.

[4] Jana Iyengar and Martin Thomson. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000, Internet Engineering Task Force, May 2021. URL https://www.rfc-editor.org/rfc/rfc9000.

[5] Tommy Pauly, Eric Kinnear, and David Schinazi. An Unreliable Datagram Extension to QUIC. RFC 9221, Internet Engineering Task Force, March 2022. URL https://www.rfc-editor.org/rfc/rfc9221.

[6] Eric Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, Internet Engineering Task Force, August 2018. URL https://www.rfc-editor.org/rfc/rfc8446.

[7] Martin Thomson and Sean Turner. Using TLS to Secure QUIC. RFC 9001, Internet Engineering Task Force, May 2021. URL https://www.rfc-editor.org/rfc/rfc9001.

[8] Lucas Clemente et al. quic-go: A quic implementation in pure go. https://github.com/quic-go/quic-go, 2016.

[9] Synadia Communications. NATS: The Connective Technology for Adaptive Edge & Distributed Systems. https://nats.io/, 2023.

Appendix A

Code Listings

Key source files are reproduced below for completeness. The full source code is available in the project repository.

A.1 Wire-Format Framing (`msgio.go`)

[Paste or `\lstinputlisting` the msgio.go source here.]

A.2 Datagram Socket (`datagram.go`)

[Paste or `\lstinputlisting` the datagram.go source here.]

Appendix B

Benchmark Raw Data

[Paste the full output of `go test -bench=. -benchtime=5s -count=5 -benchmem ./...` here.]

Appendix C

Project Timeline (Gantt Chart)

[Include a Gantt chart or table showing planned vs. actual project milestones.]

Table C.1: Project milestone summary.

Milestone	Description	Planned	Actual
M1	Transport layer complete	[date]	[date]
M2	PUB/SUB and REQ/REP complete	[date]	[date]
M3	Datagram sockets complete	[date]	[date]
M4	0-RTT and migration complete	[date]	[date]
M5	Test suite complete	[date]	[date]
M6	Benchmarks and report complete	[date]	[date]

Appendix D

Test Logs

[Paste the output of `go test -v ./...` here, or a representative excerpt showing all PASS results.]

Appendix E

Bill of Materials

As a pure software project, there is no hardware bill of materials. Table E.1 lists the software dependencies.

Table E.1: Software Bill of Materials.

Component	Version	Licence
Go toolchain	1.22	BSD 3-Clause
quic-go	v0.42.x	MIT
Go standard library	(bundled)	BSD 3-Clause